# LUCID-BB84: Syndrome-Lookup Reconciliation for BB84 with Row–Column Iterative Decoding and Block-Level Verification

Fathina U
*Department of CSE*
*School of Computing*
*SASTRA Deemed University*
Thanjavur, India
fathina0701@gmail.com

Balasubramanian TBR
*Department of CSE*
*School of Computing*
*SASTRA Deemed University*
Thanjavur, India
balasubramaniantbr@gmail.com

K. Geetha
*School of Computing*
*SASTRA Deemed University*
Thanjavur, India
geethavalavan@gmail.com

***Abstract*—Interactive Cascade reconciliation ties the latency of BB84 post-processing to the round-trip time of the classical channel. LUCID-BB84 replaces it with a one-way extended-Hamming syndrome lookup, a keyed Toeplitz verification tag, Toeplitz privacy amplification and a Cascade fallback, and its register-transfer-level front end verifies a 512-bit frame in 24 clock cycles. Because a block corrects only one error, frames must stay short, and short frames make per-frame verification costly and per-frame privacy amplification infeasible under finite-key statistics. We extend the design in three ways. LUCID-2D arranges each frame as a grid, sends row and column syndromes in the same single message, and decodes by iterating the same lookup along rows and columns entirely on syndromes, with a communication-free stall-resolution step and a one-round second-syndrome fallback. A block-level pipeline keeps only a short routing tag per frame and moves parameter estimation, the correctness tag and privacy amplification to blocks of about one million bits. A zero-leakage estimator recovers the error rate from syndromes that are already public. In a unified simulator that counts every disclosed bit, LUCID-2D reaches 94 to 98 percent of the secret fraction of a belief-propagation LDPC decoder at 0.1 to 0.5 percent error rate in about one round trip, against some 33 round trips for Cascade, and its verified decoder core uses 4 to 13 percent of a 25k-LUT FPGA at 61 to 84 MHz. At 1 percent error rate and above, LDPC remains the better choice.**

***Index Terms*—quantum key distribution, BB84, information reconciliation, product codes, Hamming code, LDPC, Cascade, privacy amplification, finite-key analysis, FPGA**

## I. Introduction

Quantum key distribution (QKD) leaves the two parties with correlated but not identical raw keys. Classical post-processing—sifting, parameter estimation, information reconciliation, verification and privacy amplification (PA)—turns them into identical keys about which an eavesdropper knows essentially nothing [1]–[3]. Reconciliation dominates both the information disclosed to the eavesdropper and the latency. Cascade [4], [5] is close to the Shannon limit but interactive, so its latency grows with the number of public-channel round trips. One-way low-density parity-check (LDPC) syndromes [6]–[8] need a single message, at the price of an iterative belief-propagation (BP) decoder [9], [10].

LUCID-BB84 takes the opposite extreme and reduces reconciliation to a table lookup. For every small extended-Hamming block [11], Alice publishes its short syndrome; Bob corrects single errors by lookup and flags double errors; a keyed Toeplitz tag verifies the frame; and frames that fail are handed to Cascade. The original study showed that this cuts round trips sharply relative to Cascade at low quantum bit error rate (QBER), but that it does not match one-way LDPC on secret-key rate, and that its main remaining argument—small, fixed-latency hardware—was untested.

Three weaknesses of the one-dimensional design limit it. First, a block corrects at most one error, so the probability that a whole frame decodes falls exponentially with frame length and frames must stay short. Second, with short frames a 64-bit tag per frame costs $64/512 = 12.5\%$ of the key, more than the reconciliation itself at 0.1% QBER. Third, estimating the phase-error rate and performing PA per 512-bit frame cannot yield a positive key once finite-size statistics are included, as the bound used in Section IV-E shows [12].

This paper keeps the lookup decoder and removes these limits. Its contributions are:

1) The LUCID front end: a bit-exact golden model, register-transfer-level (RTL) hardware with measured cycle counts, and a comparison with Cascade and one-way LDPC in which every disclosed bit is counted.
2) LUCID-2D, which places each frame on a grid, sends row and column syndromes in one message, and iterates the same lookup along rows and columns. The decoder operates only on syndromes, adds a communication-free stall-resolution step and a one-round second-syndrome fallback, and is verified and synthesized for field-programmable gate arrays (FPGAs).
3) A block-level pipeline that keeps a short routing tag per frame and moves parameter estimation, the correctness tag and PA to large blocks, together with a zero-leakage QBER estimator computed from the disclosed syndromes.
4) An evaluation in a single simulator that reproduces the

original comparison and then compares all schemes at matched frame sizes, including finite-key effects, burst noise and an ablation of each mechanism.

Related work is reviewed in Section II and the system model in Section III. Section IV presents the architecture, Section V the evaluation method and Section VI the results; Sections VII–IX discuss them, state limitations and conclude.

## II. Background and Related Work

### A. BB84 Post-Processing

After sifting, BB84 [1] leaves strings $\mathbf{a}$ and $\mathbf{b}$ whose mismatch rate is the QBER $Q$. A public sample estimates the phase-error rate; reconciliation removes the mismatches while disclosing $\ell_{\mathrm{EC}}$ bits; a universal hash [13], [14] confirms equality; and PA with a two-universal hash removes the eavesdropper's information [15]–[17]. Finite-key analyses [12], [18] subtract statistical-deviation and failure-probability terms, and Tsurumaru *et al.* showed that the verification step must be reflected explicitly in the secrecy definition [19].

### B. Reconciliation Protocols

Cascade exchanges block parities over several passes with binary search and back-tracking [4]; its efficiency and its many rounds are analysed in [5]. Winnow [20] introduced Hamming syndromes to QKD reconciliation: parities are compared first and syndromes are then exchanged for the blocks whose parities differ, so the protocol remains interactive. One-way LDPC reconciliation [7], [8] is standard in high-rate systems, with rate-adaptive and blind variants [21], [22] and high-throughput implementations [23], [24]. Kiktenko *et al.* estimated the QBER from LDPC syndromes [25]; our estimator applies the same idea to extended-Hamming syndromes in closed form. Finite-length limits of one-way reconciliation are given in [26].

### C. Product Codes

Product codes [27] arrange data in a grid whose rows and columns are codewords. Soft iterative decoding gives block turbo codes [28]; hard-decision iterative decoding of product-like codes with algebraic components underlies high-speed optical forward error correction [29], [30], where miscorrections and stall patterns are known limitations [31]. We are not aware of their use for QKD reconciliation in syndrome form, or of a decoder that runs purely on syndrome differences as described here.

### D. Hardware Post-Processing and Applications

FPGA key-distillation engines exist [32], and smart-grid links are a motivating application with short, low-loss fibre and modest QBER [33]. The smart-grid setting serves here only to fix the range of QBER studied (0.1–1%); no field result is claimed.

## III. System Model and Assumptions

Every result in this paper comes from simulation and rests on these assumptions.

- **A1** The source emits ideal single photons; channel loss, dark counts and detector side channels are not modelled.
- **A2** Independent, symmetric bit flips with probability $e$ on Bob's outcomes; Section VI-G relaxes this with a Gilbert–Elliott burst model [34], [35].
- **A3** Eve measures a fraction $p$ of the photons in a randomly chosen basis and resends the result, so that $Q = e + p/4 - ep/2$.
- **A4** A randomly chosen public sample of sifted bits provides the estimate of the phase-error rate.
- **A5** A simplified Serfling-type bound [36] with a leftover-hash term, following the structure of [12] but not its exact composable statement.
- **A6** Hash seeds and authentication keys are drawn from a physical random source or taken from previously shared secret key; seeded pseudo-random generators appear only in the simulator, for reproducibility.
- **A7** The classical channel is authenticated with a Wegman–Carter tag [37].

Under A1–A3 every scheme sees the same independent error pattern $\mathbf{e} = \mathbf{a} \oplus \mathbf{b}$, and every disclosed hint is linear in Alice's string, so simulating $\mathbf{e}$ is exact.

*Accounting.* The asymptotic secret fraction of a frame of $N$ sifted bits is

$$r = \mathbb{E}\left[\mathbf{1}_{\mathrm{ok}}\left(1 - h(Q) - \frac{\ell_{\mathrm{EC}} + \ell_{\mathrm{ver}}}{N}\right)\right], \tag{1}$$

where $h$ is the binary entropy and $\mathbf{1}_{\mathrm{ok}}$ marks frames that end identical and verified. $\ell_{\mathrm{EC}}$ counts every syndrome bit, parity and second-step syndrome. Under *frame-local* accounting (the original design) every verification discloses a 64-bit tag. Under *block-level* accounting (Section IV-E) every verification discloses a $t_r = 24$-bit routing tag and a single 64-bit correctness tag is amortized over a PA block of $n_{\mathrm{PA}} = 2^{20}$ bits. With mean round trips $\rho$ and round-trip time $T$, the secret throughput is $Nr/(\rho T + T_c)$; since the compute time $T_c$ is negligible here for all schemes, throughput is proportional to $r/\rho$.

## IV. LUCID Architecture

Fig. 1 summarizes the complete pipeline described in this section.

### A. One-Dimensional Syndrome Lookup

After sifting and sampling, a frame is split into blocks of $C = 2^m$ bits. The syndrome of block $\mathbf{x}$ has $m + 1$ bits: $s_k$ ($k < m$) is the XOR of the bits whose position has bit $k$ set, and $s_m$ is the overall parity. Alice sends all block syndromes and the tag in one message. Bob forms $\mathbf{d} = \mathbf{s}_A \oplus \mathbf{s}_B$; if $d_m = 1$ he flips position $(d_{m-1} \dots d_0)_2$, if $d_m = 0$ and $\mathbf{d} \neq \mathbf{0}$ he flags a double error, and three or more errors may be mis-corrected and are left to the tag. Table I works through one block. The disclosure is $(m + 1)/2^m$ bits per key bit regardless of $Q$.

TABLE I
WORKED EXAMPLE: $(8, 4)$ BLOCK WITH AN ERROR AT POSITION 5

| Quantity | Value |
|---|---|
| Alice block (bits 0–7) | 1 0 1 1 0 0 1 0 |
| Alice syndrome $(s_0, s_1, s_2, s_3)$ | (1, 1, 1, 0) |
| Bob block (flip at 5) | 1 0 1 1 0 1 1 0 |
| Bob syndrome | (0, 1, 0, 1) |
| $\mathbf{d} = \mathbf{s}_A \oplus \mathbf{s}_B$ | (1, 0, 1, 1) |
| Decision | $s_3$ differs: flip bit $1 + 4 = 5$ |

### B. Front-End Hardware

The front-end RTL consists of a block corrector pipelined over three stages (Bob's syndrome, decoding of the syndrome difference, correction) that accepts one block per lane and clock cycle, a Toeplitz hash engine consuming $W$ bits per cycle, a Wegman–Carter tag unit, and a controller that sequences correction, verification and PA. RTL simulation gives $2N/W+8$ cycles for a frame that verifies, which for $N = 512$ and $W = 64$ is 24 cycles or 120 ns at a 200-MHz clock, and $N/W + 6$ cycles for a frame whose tag fails. Error correction alone takes three cycles per frame.

### C. LUCID-2D: Row–Column Iterative Lookup

LUCID-2D partitions a frame into tiles of $R \times C$ bits ($R = 2^{m_r}$, $C = 2^{m_c}$). Alice discloses the extended-Hamming syndrome of every row (length $C$) and every column (length $R$) of every tile in the same single message. Because $(m_r+1)(m_c+1)$ of these bits are determined by the others, the information disclosed per tile is the rank of the product-code syndrome map,

$$\ell_{\text{tile}} = R(m_c + 1) + C(m_r + 1) - (m_r + 1)(m_c + 1), \quad (2)$$

which reduces to $m_c+1$ for $R = 1$, the one-dimensional case.

*Syndrome-domain decoding.* Let $h_n(x) = (1, x)$ denote the $(\log_2 n + 1)$-bit SECDED column of position $x$ in a block of length $n$. Bob holds only the row differences $\mathbf{d}_i^{\text{r}}$ and column differences $\mathbf{d}_j^{\text{c}}$. Correcting bit $(i, j)$ XORs $h_C(j)$ into $\mathbf{d}_i^{\text{r}}$ and $h_R(i)$ into $\mathbf{d}_j^{\text{c}}$, so the key bits never need to be read during decoding: a row pass examines each $\mathbf{d}_i^{\text{r}}$, emits a flip $(i, j)$ when its top bit is set, clears it and updates $\mathbf{d}_j^{\text{c}}$; a column pass does the converse. Decoding stops after the first row–column scan without a correction, or after nine scans; the tile has converged when that scan saw only zero syndromes. The decoder state is $R(m_c + 1) + C(m_r + 1)$ bits—4608 bits for a $256 \times 256$ tile instead of 65536—and the flip list is XORed into the key buffer.

*Stall resolution.* The dominant residual pattern is a "rectangle" of four errors at the corners of two rows and two columns: every affected line sees two errors and only flags them. Bob therefore flips every intersection of a non-zero row and a non-zero column when there are at most 16 of them and iterates again, at most twice; the limit of 16 was chosen on a separate pilot run. This needs no communication; a wrong guess either leaves non-zero syndromes or is caught by the tag.

*Fallback ladder.* If a tile still fails, Bob announces the failed tiles and the indices of their non-zero rows and columns, which concerns only the error pattern, and Alice answers in the same round trip with SECDED syndromes of 16-bit blocks taken, in a public pseudo-random order, over the *suspect bits*—the union of the non-zero rows and columns of the failed tiles (the flagged blocks in the one-dimensional case). Bob corrects them and re-runs the tile decoder. Only if a tile still fails is Cascade run, and only on the remaining suspect bits; if the frame tag then fails, Cascade runs on the whole frame.

### D. Zero-Leakage QBER Estimation

A block of length $n$ with at least one error has a non-zero syndrome difference except for patterns of weight at least four, so the fraction $\hat{p}$ of non-zero row and column differences gives $\hat{Q} = 1 - (1 - \hat{p})^{1/n}$. Bob computes $\hat{Q}$ before decoding with a population count over syndromes that are already public, so it costs no key. It drives the choice of tile shape for the next frame; it does *not* replace the random sample that bounds the phase-error rate in the security analysis (A4), which is taken once per PA block.

### E. Block-Level Verification and Privacy Amplification

Per-frame PA on $N = 512$ bits cannot produce key under A5: with a 64-bit sample the deviation term is

$$\mu = \sqrt{\frac{(n + k)(k + 1) \ln(1/\varepsilon_{\text{PE}})}{2nk^2}}, \quad (3)$$

which equals 0.45 for $n = 512$, $k = 64$ and $\varepsilon_{\text{PE}} = 10^{-10}$, far above the 11% threshold. We therefore reconcile per frame and accumulate verified frames into PA blocks of $n$ bits, for which

$$\ell = n\big[1 - h(Q + \mu)\big] - \ell_{\text{EC}} - \ell_{\text{ver}} - 2\log_2 \frac{1}{\varepsilon_{\text{PA}}}. \quad (4)$$

Verification has two tiers: a $t_r$-bit routing tag per frame only decides whether to fall back, and one 64-bit tag per PA block provides the correctness guarantee. A wrong frame passes the routing tag with probability $2^{-t_r}$, in which case the block tag rejects the whole block; with $t_r = 24$ and up to $2^{11}$ frames per block this happens with probability at most $2^{-13}$ even if every frame were wrong after decoding. Authenticating the transcript once per block with a Wegman–Carter tag [37] likewise avoids spending one-time-pad key on every Cascade round.

## V. METHODOLOGY

### A. Simulator and Schemes

A single Python simulator implements all schemes on the error pattern. *Cascade* [4] uses four passes with doubling blocks, binary search and back-tracking; all searches that can proceed in parallel do so, and one parity request–response is one round trip, so the reported rounds are a lower bound. *LUCID (1-D)* uses blocks of $C \in \{8, \ldots, 1024\}$ bits with the original whole-frame fallback at $N = 512$ and the fallback ladder elsewhere. *LUCID-2D* uses tiles with $8 \le R \le C \le 1024$. *BP-LDPC* uses a random code with variable degree three, distinct checks

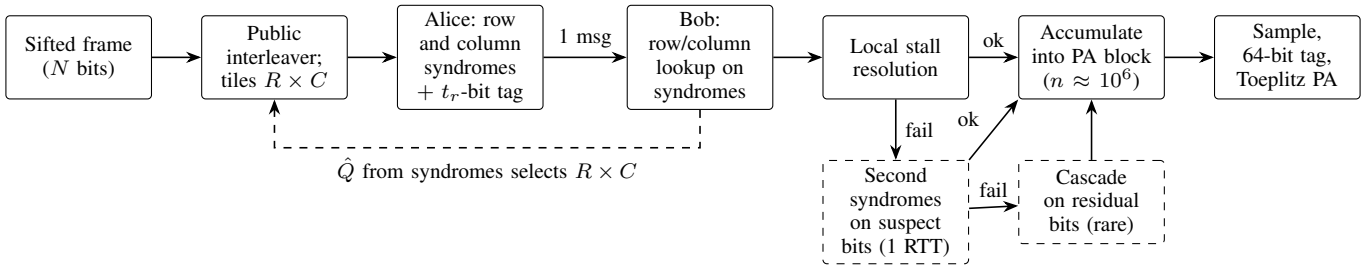



Fig. 1. Reconciliation pipeline with LUCID-2D, the fallback ladder (dashed boxes) and block-level privacy amplification. Only the first message is needed when a frame decodes; the dashed arrow is the syndrome-based QBER estimate that selects the tile shape of the next frame.

per column and balanced check degrees, $m = \lceil f\, h(Q)N \rceil$ syndrome bits, and sum-product decoding [10] with up to 100 iterations; failed frames are lost. Cascade and LDPC are given the true QBER, which favours them.

*Parameter selection.* For every scheme, frame size and QBER, the parameter (block size, tile shape, Cascade first block $k_1$, or $f$) is chosen on a pilot run and the reported values come from an independent run with fresh seeds. At $N = 512$ the criterion is the largest $r$, as in the original study. At matched frame sizes, LUCID variants maximize $r$ subject to $\rho \leq 1.1$ ("one round"), or $r/\rho$ when no parameter meets this budget; Cascade maximizes $r/\rho$; and LDPC, which always needs one round, maximizes $r$. Final runs use 2000 frames at $N = 512$ and 400 frames (LUCID) or 100 frames (Cascade, LDPC) at $N = 16384$ and 65536; the 95% confidence half-width of $r$ is at most 0.011 at $N = 512$ and 0.002 at $N = 65536$.

### B. Hardware Flow

The LUCID-2D decoder core is written in Verilog, checked cycle by cycle against a Python model of the syndrome-domain decoder with Icarus Verilog [38] (flips, convergence, scan and cycle counts; zero mismatches over 520 random tiles of six shapes), and synthesized with Yosys and placed and routed with nextpnr [39]. Stall resolution and the fallback ladder are rare control paths and are not part of the core.

## VI. Results

### A. Validation and Reproduction

The photon-level BB84 simulator matches $Q = e + p/4 - ep/2$ to within 0.0011 at all five tested $(e, p)$ points with $3 \times 10^5$ qubits. Table II reproduces the original $N = 512$ comparison under frame-local accounting. The secret fractions agree with the original golden model to within 0.045 and rank the three schemes identically at every QBER. At this size the one-dimensional hybrid trails Cascade in $r$, lies above the unoptimized LDPC code up to 0.5% QBER and below it at 1% and 2%, and the 64-bit tag alone costs 0.125 of every key bit. Choosing the block size for throughput instead of $r$, it delivers 6.0, 5.6 and 3.1 times the round-trip-limited secret throughput of Cascade at 0.1, 0.2 and 0.5% QBER and falls below Cascade at 2% (0.71 times), consistent with the original study.

TABLE II
Secret Fraction $r$ and Mean Round Trips $\rho$ at $N = 512$, Frame-Local Accounting (64-bit Tag per Verification)

| | Cascade | | LUCID (1-D) | | BP-LDPC | |
|---|---|---|---|---|---|---|
| QBER (%) | $r$ | $\rho$ | $r$ | $\rho$ | $r$ | $f$ |
| 0.1 | 0.828 | 6.53 | 0.818 | 1.48 | 0.750 | 6.0 |
| 0.2 | 0.807 | 8.09 | 0.782 | 2.84 | 0.723 | 5.0 |
| 0.5 | 0.744 | 9.70 | 0.647 | 4.18 | 0.634 | 4.0 |
| 1.0 | 0.675 | 12.4 | 0.456 | 14.9 | 0.543 | 2.5 |
| 2.0 | 0.551 | 19.3 | 0.378 | 24.4 | 0.422 | 2.0 |

### B. One-Round Decoding Success

Fig. 2 compares lookup-only decoding (no fallback) at equal frame size and equal row code. With 128-bit one-dimensional blocks, a 16384-bit frame decodes with probability 0.373 at 0.1% QBER and 0.031 at 0.2%; with 256-bit blocks, a 65536-bit frame decodes in 0.05% of trials at 0.1% QBER. Adding the column syndromes changes this qualitatively: with stall resolution, the $128 \times 128$ grid decodes every one of 2000 frames at 0.2% and 97.7% at 0.5%, and the $256 \times 256$ grid decodes 99.6% at 0.2%. Stall resolution raises success where stalls dominate—from 0.894 to 0.977 for $128 \times 128$ at 0.5%, and from 0.697 to 0.859 for $256 \times 256$ at 0.3%—but cannot help once many lines fail, because it is limited to 16 candidate intersections. Tiles need on average 1.0–2.7 row–column iterations at QBER up to 0.2%.

### C. Matched Frame Size

Table III compares all schemes at $N = 65536$ under block-level accounting, and Fig. 3 shows the secret fraction against round trips for every configuration in a separate sweep. At 0.1% and 0.2% QBER the selected configuration is a single $256 \times 256$ tile per frame. It reaches $r = 0.919$ and 0.910, that is 96.3% and 98.2% of BP-LDPC (0.954 and 0.927), in $\rho = 1.00$ round trips. Cascade attains the highest secret fraction (0.974 and 0.953) but needs 33 round trips, so when the round-trip time dominates, LUCID-2D delivers 31 times the secret throughput of Cascade. The one-dimensional lookup cannot do the same: within one round its best option at 0.2% discloses half of the key ($r = 0.478$ with 8-bit blocks), and longer blocks trigger fallbacks on most frames. At 0.5% LUCID-2D reaches 0.813 with $64 \times 256$ tiles at $\rho = 1.12$ (94% of BP-LDPC); at 1% and 2% it falls to 85% and 52% of BP-LDPC,

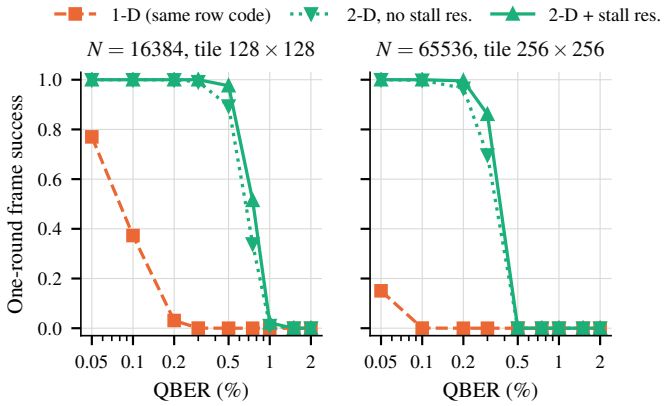



Fig. 2. Probability that a frame decodes with the lookup alone (no fallback), for one-dimensional blocks and for a single square tile with the same row code.

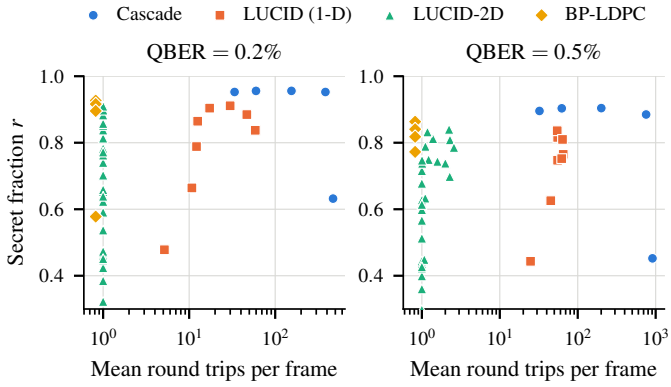



Fig. 3. Secret fraction versus mean round trips per frame for every parameter in a sweep at $N = 65536$ (block-level accounting). All LDPC points have exactly one round and are shifted left for visibility.

which is then clearly preferable. Fig. 3 shows that LUCID-2D shares the one-round corner of the trade-off only with LDPC, while Cascade and the fallback-heavy one-dimensional configurations reach comparable secret fractions only with tens to hundreds of round trips. At $N = 16384$ the picture is the same, with LUCID-2D at 92.9–99.4% of BP-LDPC up to 0.5% QBER.

### D. Ablation

Table IV isolates stall resolution and the second-syndrome step. Without either mechanism a $256 \times 256$ tile at 0.2% QBER falls back in 3.7% of frames, and each fallback costs about 20 Cascade round trips ($\rho = 1.76$). Stall resolution alone removes most fallbacks (0.3%, $\rho = 1.15$); the second syndrome message alone makes the remaining ones cheap ($\rho = 1.03$); together they give $\rho = 1.00$. At 0.5% the two are complementary, reducing $\rho$ from 11.5 to 1.24 for $128 \times 128$ tiles. At 1% with $64 \times 64$ tiles the second message alone is slightly better than the combination (1.85 versus 2.09), because stall-resolution guesses in tiles with many failed lines can add errors. Both mechanisms disclose so little that $r$ changes by at most 0.003.

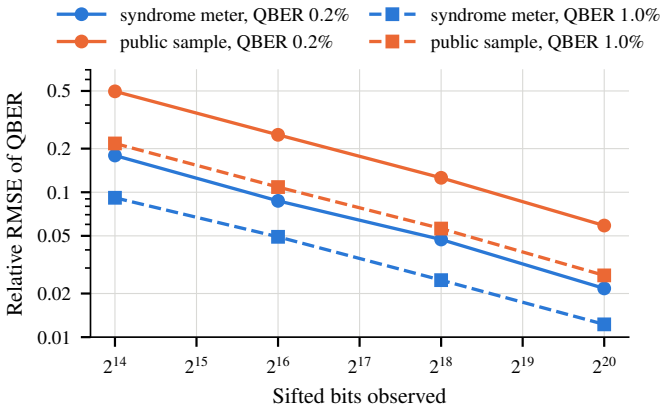



Fig. 4. Relative root-mean-square error of the QBER estimate versus the number of sifted bits observed: syndrome meter on $128 \times 128$ tiles (no disclosure) and a public sample of 64 bits per 512 key bits (400 trials per point).

### E. QBER Estimation and Closed-Loop Shape Selection

Fig. 4 compares the syndrome meter with the public sample rate of the original design, 64 sample bits per 512 key bits. Over the same observation window the syndrome meter on $128 \times 128$ tiles has 2.7–2.9 times lower relative error at 0.2% QBER and 2.2–2.4 times lower at 1%, while disclosing nothing; the sample in addition consumes one sifted bit in nine. A single 64-bit sample per 512-bit frame, as in the original design, contains no error at all in 88% of frames at 0.2% QBER, so it cannot steer per-frame choices. In a closed-loop run of 600 frames of 65536 bits whose QBER drifts between 0.1% and 1%, choosing the tile shape from the syndrome estimate gives $r = 0.821$ at $\rho = 1.22$, matching a controller that knows the true QBER (0.821, $\rho = 1.17$). A sample-driven controller reaches 0.728 because of the sacrificed bits, and a fixed shape tuned for 0.2% needs $\rho = 14.8$.

### F. Finite-Key Effects

Fig. 5 applies (4) with the leakage measured at $N = 65536$, optimizing the sample size $k$ for each block size. Per-frame PA on 512 bits yields no key at any QBER (crosses). With block-level PA every scheme approaches its asymptotic value: at 0.2% QBER and $n = 2^{20}$ the finite-key fraction is 0.808 for Cascade, 0.783 for BP-LDPC and 0.767 for LUCID-2D, rising to 0.910, 0.885 and 0.868 at $n = 2^{26}$. The finite-size penalty is common to all schemes, so their order is preserved; the gain over the original pipeline comes from replacing the per-frame 64-bit tag and per-frame PA.

### G. Burst Noise

Table V uses a Gilbert–Elliott channel [34], [35] with no errors in the good state, error probability 0.25 in the bad state and the stationary QBER held fixed, using the tiles selected in Table III. When tiles are filled row by row, bursts of mean length 8 place several errors in the same row and raise the fallback rate at 0.2% QBER from zero to 93% ($\rho = 4.25$). A fixed public pseudo-random interleaver applied before tiling

TABLE III
SECRET FRACTION $r$ AND MEAN ROUND TRIPS $\rho$ AT $N = 65536$, BLOCK-LEVEL ACCOUNTING. LUCID PARAMETERS WERE SELECTED FOR $\rho \leq 1.1$ ON A PILOT RUN WHERE POSSIBLE

| | Cascade | | LUCID (1-D) | | LUCID-2D | | | BP-LDPC ($\rho = 1$) | |
|---|---|---|---|---|---|---|---|---|---|
| QBER (%) | $r$ | $\rho$ | $r$ | $\rho$ | tile | $r$ | $\rho$ | $r$ | $f$ |
| 0.1 | 0.974 | 33.3 | 0.800 | 4.45 | 256×256 | 0.919 | 1.00 | 0.954 | 3.0 |
| 0.2 | 0.953 | 33.3 | 0.478 | 5.38 | 256×256 | 0.910 | 1.00 | 0.927 | 2.5 |
| 0.5 | 0.896 | 33.8 | 0.444 | 23.2 | 64×256 | 0.813 | 1.12 | 0.863 | 2.0 |
| 1.0 | 0.812 | 30.8 | 0.681 | 64.8 | 32×64 | 0.642 | 1.09 | 0.757 | 2.0 |
| 2.0 | 0.664 | 28.1 | 0.550 | 64.2 | 16×16 | 0.331 | 1.21 | 0.632 | 1.6 |

TABLE IV
ABLATION AT $N = 65536$: MEAN ROUND TRIPS $\rho$ WITHOUT AND WITH STALL RESOLUTION (SR) AND THE SECOND SYNDROME MESSAGE (2S). THE SECRET FRACTION $r$ (SHOWN FOR SR + 2S) VARIES BY AT MOST 0.003 ACROSS THE FOUR VARIANTS

| | | | $\rho$ | | | |
|---|---|---|---|---|---|---|
| QBER (%) | Tile | $r$ | neither | SR | 2S | SR + 2S |
| 0.2 | 256×256 | 0.910 | 1.76 | 1.15 | 1.03 | 1.00 |
| 0.5 | 64×128 | 0.789 | 4.38 | 1.21 | 1.19 | 1.00 |
| 0.5 | 128×128 | 0.833 | 11.5 | 4.05 | 1.38 | 1.24 |
| 1.0 | 32×64 | 0.642 | 11.1 | 2.15 | 1.53 | 1.11 |
| 1.0 | 64×64 | 0.710 | 26.3 | 10.7 | 1.85 | 2.09 |

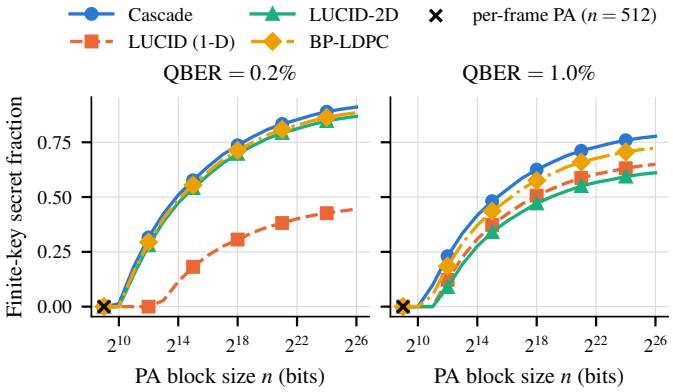



Fig. 5. Finite-key secret fraction $\ell/(n+k)$ versus PA block size, with the sample size optimized; $\varepsilon_{\mathrm{PE}} = \varepsilon_{\mathrm{PA}} = 10^{-10}$. The cross marks per-frame PA of the original design.

spreads the bursts: the fallback rate returns to zero for $L = 8$ and stays at or below 14% at 0.2% and 35% at 0.5% for $L = 128$, with $\rho \leq 1.44$. The interleaver adds neither disclosure nor communication.

### H. Hardware

Table VI lists post-route results for the decoder core on an ECP5-25F. The syndrome memories map to distributed RAM, so logic grows slowly with tile size; the $256 \times 256$ core selected at 0.1–0.2% QBER uses 13.1% of the device and closes at 61 MHz. For the AMD 7-series family, synthesis alone gives 143 logic LUTs, 48 `RAM64M` primitives and 43 flip-flops for the same core. A scan takes $R+C$ cycles; with the 2.7 iterations measured at 0.2% QBER plus the final check, a 65536-bit tile is decoded in about 1900 cycles, 31 µs at 61 MHz, or 2.1 Gbit/s of sifted key per core—less than the round-trip time of any fibre link longer than about 3 km. Computing the syndromes adds $N/W$ cycles in the front end. On an iCE40 HX8K, which has no distributed RAM, the $64 \times 128$ core needs 3953 logic cells at 37.8 MHz and the $256 \times 256$ core does not fit. By comparison, a BP decoder for the same frame stores one message per edge, $3N = 196{,}608$ messages or about 1.2 Mbit at an assumed 6-bit quantization, and our BP decoder used 7–13 iterations on average, against 4.6 kbit of syndrome state and 1.1–3.9 row–column iterations for LUCID-2D.

TABLE V
FALLBACK RATE AND MEAN ROUND TRIPS OF LUCID-2D UNDER GILBERT–ELLIOTT NOISE WITH MEAN BURST LENGTH $L$

| | | Row-major fill | | Interleaved fill | |
|---|---|---|---|---|---|
| QBER (%) | $L$ | fallback | $\rho$ | fallback | $\rho$ |
| 0.2 | 1 | 0.00 | 1.00 | 0.01 | 1.01 |
| 0.2 | 8 | 0.93 | 4.25 | 0.00 | 1.00 |
| 0.2 | 32 | 0.89 | 4.77 | 0.05 | 1.05 |
| 0.2 | 128 | 0.67 | 6.47 | 0.14 | 1.14 |
| 0.5 | 1 | 0.16 | 1.77 | 0.12 | 1.12 |
| 0.5 | 8 | 0.99 | 3.21 | 0.12 | 1.12 |
| 0.5 | 32 | 0.98 | 4.29 | 0.19 | 1.19 |
| 0.5 | 128 | 0.96 | 26.8 | 0.35 | 1.44 |

TABLE VI
LUCID-2D DECODER CORE AFTER PLACE AND ROUTE ON A LATTICE ECP5 LFE5U-25F (YOSYS AND NEXTPNR)

| Tile | LUT4 | DPRAM | FF | Logic cells used | $f_{\max}$ (MHz) |
|---|---|---|---|---|---|
| 64 × 128 | 562 | 48 | 38 | 864 (3.6%) | 84.0 |
| 128 × 128 | 536 | 64 | 39 | 934 (3.8%) | 76.2 |
| 256 × 256 | 2004 | 192 | 43 | 3170 (13.1%) | 61.1 |

## VII. DISCUSSION

*Where LUCID-2D fits.* In the low-QBER regime targeted by the original design (0.1–0.5%), LUCID-2D reaches 94–98% of the secret fraction of an oracle-tuned BP decoder within about one round trip, and 27–32 times the round-trip-limited throughput of Cascade. Its decoder is a hard-decision lookup that needs no estimate of the QBER to decode; the syndrome meter only selects the tile shape. Above about 1% QBER the fixed disclosure of extended-Hamming components makes it uncompetitive, and LDPC or Cascade should be used.

*Fairness of the comparison.* The LDPC baseline is an unoptimized random code; optimized codes reach efficiencies close to 1.1 at long frames [7], [8], [24] and would widen its lead, particularly above 0.5%. On the other hand it is given the true QBER and the best syndrome length, and its decoding time is ignored. Cascade is given the true QBER and an optimized first block, and its rounds are a lower bound.

*Pipeline.* The block-level pipeline helps every scheme, not only LUCID: at 0.1% QBER Cascade rises from 0.828 at $N = 512$ to 0.974 at $N = 65536$, mainly because the per-frame 64-bit tag disappears. If each public message were authenticated with its own one-time-pad tag, interactive protocols would also spend key on every round trip; authenticating the transcript once per block avoids this for all schemes.

*Smart grid.* Key refresh is not latency-critical, so for such links the value of LUCID-2D lies in fewer messages and a small, fixed-structure decoder at edge devices rather than in latency. An optional decode cache studied for the original front end is omitted: the lookup already completes in one cycle, and caching key-dependent blocks would keep secret material in hardware longer than necessary.

*Threats to validity.* The noise models are independent flips and a synthetic burst model; the finite-key bound is simplified (A5); Cascade and LDPC compute times are ignored; the front-end RTL was characterized in simulation but not re-synthesized; stall resolution, the second syndrome step and the interleaver are not part of the RTL core. Confidence half-widths of $r$ are at most 0.002 at $N = 65536$ and 0.011 at $N = 512$.

## VIII. Limitations and Future Work

- The source is ideal (A1); a decoy-state analysis [18] with realistic loss and detectors is needed for deployment figures.
- The finite-key bound is not the exact composable statement of [12], and the treatment of verification should follow [19].
- No power, ASIC or AMD timing results are reported, and no LDPC decoder was synthesized on the same device.
- Future work: a composable decoy-state analysis; moving stall resolution and the interleaver into the RTL; BCH component codes or staircase arrangements [30] to raise efficiency above 0.5% QBER; and an optimized quasi-cyclic LDPC baseline synthesized on the same FPGA.

## IX. Conclusion

LUCID-BB84 reconciles BB84 keys with a one-way syndrome lookup and a verified fallback, and its front end verifies a 512-bit frame in 120 ns. Arranging frames as grids of extended-Hamming codes, decoding iteratively in the syndrome domain, resolving stalls locally and falling back with a single extra syndrome message turns this lookup into a one-round reconciler that reaches 94–98% of the secret fraction of BP-LDPC at 0.1–0.5% QBER, with 27–32 times the round-trip-limited throughput of Cascade. Moving verification and privacy amplification to blocks of about a million bits makes the finite-key rate positive and lets it approach the asymptotic rate as blocks grow, and the syndromes themselves provide a free, accurate QBER estimate. The decoder core is verified against its model and fits in 4–13% of a 25k-LUT FPGA at 61–84 MHz. Above 1% QBER, one-way LDPC remains the better choice.

## References


[1] C. H. Bennett and G. Brassard, "Quantum cryptography: Public key distribution and coin tossing," in *Proc. IEEE Int. Conf. Comput., Syst. Signal Process.*, Bangalore, India, Dec. 1984, pp. 175–179.

[2] V. Scarani, H. Bechmann-Pasquinucci, N. J. Cerf, M. Dušek, N. Lütkenhaus, and M. Peev, "The security of practical quantum key distribution," *Rev. Mod. Phys.*, vol. 81, no. 3, pp. 1301–1350, 2009.

[3] S. Pirandola, U. L. Andersen, L. Banchi, M. Berta, D. Bunandar, R. Colbeck, D. Englund, T. Gehring, C. Lupo, C. Ottaviani, J. L. Pereira, M. Razavi, J. S. Shaari, M. Tomamichel, V. C. Usenko, G. Vallone, P. Villoresi, and P. Wallden, "Advances in quantum cryptography," *Adv. Opt. Photon.*, vol. 12, no. 4, pp. 1012–1236, 2020.

[4] G. Brassard and L. Salvail, "Secret-key reconciliation by public discussion," in *Advances in Cryptology—EUROCRYPT '93*, ser. Lecture Notes in Computer Science, vol. 765. Berlin, Germany: Springer, 1994, pp. 410–423.

[5] J. Martinez-Mateo, C. Pacher, M. Peev, A. Ciurana, and V. Martin, "Demystifying the information reconciliation protocol Cascade," *Quantum Inf. Comput.*, vol. 15, no. 5–6, pp. 453–477, 2015.

[6] R. G. Gallager, "Low-density parity-check codes," *IRE Trans. Inf. Theory*, vol. 8, no. 1, pp. 21–28, 1962.

[7] D. Elkouss, A. Leverrier, R. Alleaume, and J. J. Boutros, "Efficient reconciliation protocol for discrete-variable quantum key distribution," in *Proc. IEEE Int. Symp. Inf. Theory (ISIT)*, Seoul, South Korea, 2009, pp. 1879–1883.

[8] D. Elkouss, J. Martinez-Mateo, and V. Martin, "Information reconciliation for quantum key distribution," *Quantum Inf. Comput.*, vol. 11, no. 3–4, pp. 226–238, 2011.

[9] D. J. C. MacKay, "Good error-correcting codes based on very sparse matrices," *IEEE Trans. Inf. Theory*, vol. 45, no. 2, pp. 399–431, 1999.

[10] F. R. Kschischang, B. J. Frey, and H.-A. Loeliger, "Factor graphs and the sum-product algorithm," *IEEE Trans. Inf. Theory*, vol. 47, no. 2, pp. 498–519, 2001.

[11] R. W. Hamming, "Error detecting and error correcting codes," *Bell Syst. Tech. J.*, vol. 29, no. 2, pp. 147–160, 1950.

[12] M. Tomamichel, C. C. W. Lim, N. Gisin, and R. Renner, "Tight finite-key analysis for quantum cryptography," *Nat. Commun.*, vol. 3, no. 1, Art. no. 634, 2012.

[13] J. L. Carter and M. N. Wegman, "Universal classes of hash functions," *J. Comput. Syst. Sci.*, vol. 18, no. 2, pp. 143–154, 1979.

[14] H. Krawczyk, "LFSR-based hashing and authentication," in *Advances in Cryptology—CRYPTO '94*, ser. Lecture Notes in Computer Science, vol. 839. Berlin, Germany: Springer, 1994, pp. 129–139.

[15] C. H. Bennett, G. Brassard, C. Crépeau, and U. M. Maurer, "Generalized privacy amplification," *IEEE Trans. Inf. Theory*, vol. 41, no. 6, pp. 1915–1923, 1995.

[16] R. Renner, "Security of quantum key distribution," *Int. J. Quantum Inf.*, vol. 6, no. 1, pp. 1–127, 2008.

[17] P. W. Shor and J. Preskill, "Simple proof of security of the BB84 quantum key distribution protocol," *Phys. Rev. Lett.*, vol. 85, no. 2, pp. 441–444, 2000.

[18] C. C. W. Lim, M. Curty, N. Walenta, F. Xu, and H. Zbinden, "Concise security bounds for practical decoy-state quantum key distribution," *Phys. Rev. A*, vol. 89, no. 2, Art. no. 022307, 2014.

[19] T. Tsurumaru, A. Mizutani, and T. Sasaki, "Security loophole in error verification in quantum key distribution," *Phys. Rev. Res.*, vol. 7, no. 4, Art. no. 043245, 2025.

[20] W. T. Buttler, S. K. Lamoreaux, J. R. Torgerson, G. H. Nickel, C. H. Donahue, and C. G. Peterson, "Fast, efficient error reconciliation for quantum cryptography," *Phys. Rev. A*, vol. 67, no. 5, Art. no. 052303, 2003.

[21] J. Martinez-Mateo, D. Elkouss, and V. Martin, "Blind reconciliation," *Quantum Inf. Comput.*, vol. 12, no. 9–10, pp. 791–812, 2012.

[22] E. O. Kiktenko, A. S. Trushechkin, C. C. W. Lim, Y. V. Kurochkin, and A. K. Fedorov, "Symmetric blind information reconciliation for quantum key distribution," *Phys. Rev. Appl.*, vol. 8, no. 4, Art. no. 044017, 2017.

[23] A. R. Dixon and H. Sato, "High speed and adaptable error correction for megabit/s rate quantum key distribution," *Sci. Rep.*, vol. 4, no. 1, Art. no. 7275, 2014.

[24] H. Mao, Q. Li, Q. Han, and H. Guo, "High-throughput and low-cost LDPC reconciliation for quantum key distribution," *Quantum Inf. Process.*, vol. 18, no. 7, Art. no. 232, 2019.

[25] E. O. Kiktenko, A. O. Malyshev, A. A. Bozhedarov, N. O. Pozhar, M. N. Anufriev, and A. K. Fedorov, "Error estimation at the information reconciliation stage of quantum key distribution," *J. Russ. Laser Res.*, vol. 39, no. 6, pp. 558–567, 2018.

[26] M. Tomamichel, J. Martinez-Mateo, C. Pacher, and D. Elkouss, "Fundamental finite key limits for one-way information reconciliation in quantum key distribution," *Quantum Inf. Process.*, vol. 16, no. 11, Art. no. 280, 2017.

[27] P. Elias, "Error-free coding," *Trans. IRE Prof. Group Inf. Theory*, vol. 4, no. 4, pp. 29–37, 1954.

[28] R. M. Pyndiah, "Near-optimum decoding of product codes: Block turbo codes," *IEEE Trans. Commun.*, vol. 46, no. 8, pp. 1003–1010, 1998.

[29] J. Justesen, "Performance of product codes and related structures with iterated decoding," *IEEE Trans. Commun.*, vol. 59, no. 2, pp. 407–415, 2011.

[30] B. P. Smith, A. Farhood, A. Hunt, F. R. Kschischang, and J. Lodge, "Staircase codes: FEC for 100 Gb/s OTN," *J. Lightw. Technol.*, vol. 30, no. 1, pp. 110–117, 2012.

[31] C. Häger and H. D. Pfister, "Approaching miscorrection-free performance of product codes with anchor decoding," *IEEE Trans. Commun.*, vol. 66, no. 7, pp. 2797–2808, 2018.

[32] J. Constantin, R. Houlmann, N. Preyss, N. Walenta, H. Zbinden, P. Junod, and A. Burg, "An FPGA-based 4 Mbps secret key distillation engine for quantum key distribution systems," *J. Signal Process. Syst.*, vol. 86, no. 1, pp. 1–15, 2017.

[33] M. Alshowkan, P. G. Evans, M. Starke, D. Earl, and N. A. Peters, "Authentication of smart grid communications using quantum key distribution," *Sci. Rep.*, vol. 12, no. 1, Art. no. 12731, 2022.

[34] E. N. Gilbert, "Capacity of a burst-noise channel," *Bell Syst. Tech. J.*, vol. 39, no. 5, pp. 1253–1265, 1960.

[35] E. O. Elliott, "Estimates of error rates for codes on burst-noise channels," *Bell Syst. Tech. J.*, vol. 42, no. 5, pp. 1977–1997, 1963.

[36] R. J. Serfling, "Probability inequalities for the sum in sampling without replacement," *Ann. Statist.*, vol. 2, no. 1, pp. 39–48, 1974.

[37] M. N. Wegman and J. L. Carter, "New hash functions and their use in authentication and set equality," *J. Comput. Syst. Sci.*, vol. 22, no. 3, pp. 265–279, 1981.

[38] S. Williams. Icarus Verilog. Accessed: Oct. 16, 2026. [Online]. Available: https://steveicarus.github.io/iverilog/

[39] D. Shah, E. Hung, C. Wolf, S. Bazanski, D. Gisselquist, and M. Milanovic, "Yosys+nextpnr: An open source framework from Verilog to bitstream for commercial FPGAs," in *Proc. IEEE 27th Annu. Int. Symp. Field-Programmable Custom Comput. Mach. (FCCM)*, San Diego, CA, USA, 2019, pp. 1–4.